

ARCOS Group

uc3m | Universidad **Carlos III** de Madrid

L3: Fundamentals of assembler programming (4) **Computer Structure**

Bachelor in Computer Science and Engineering

Bachelor in Applied Mathematics and Computing

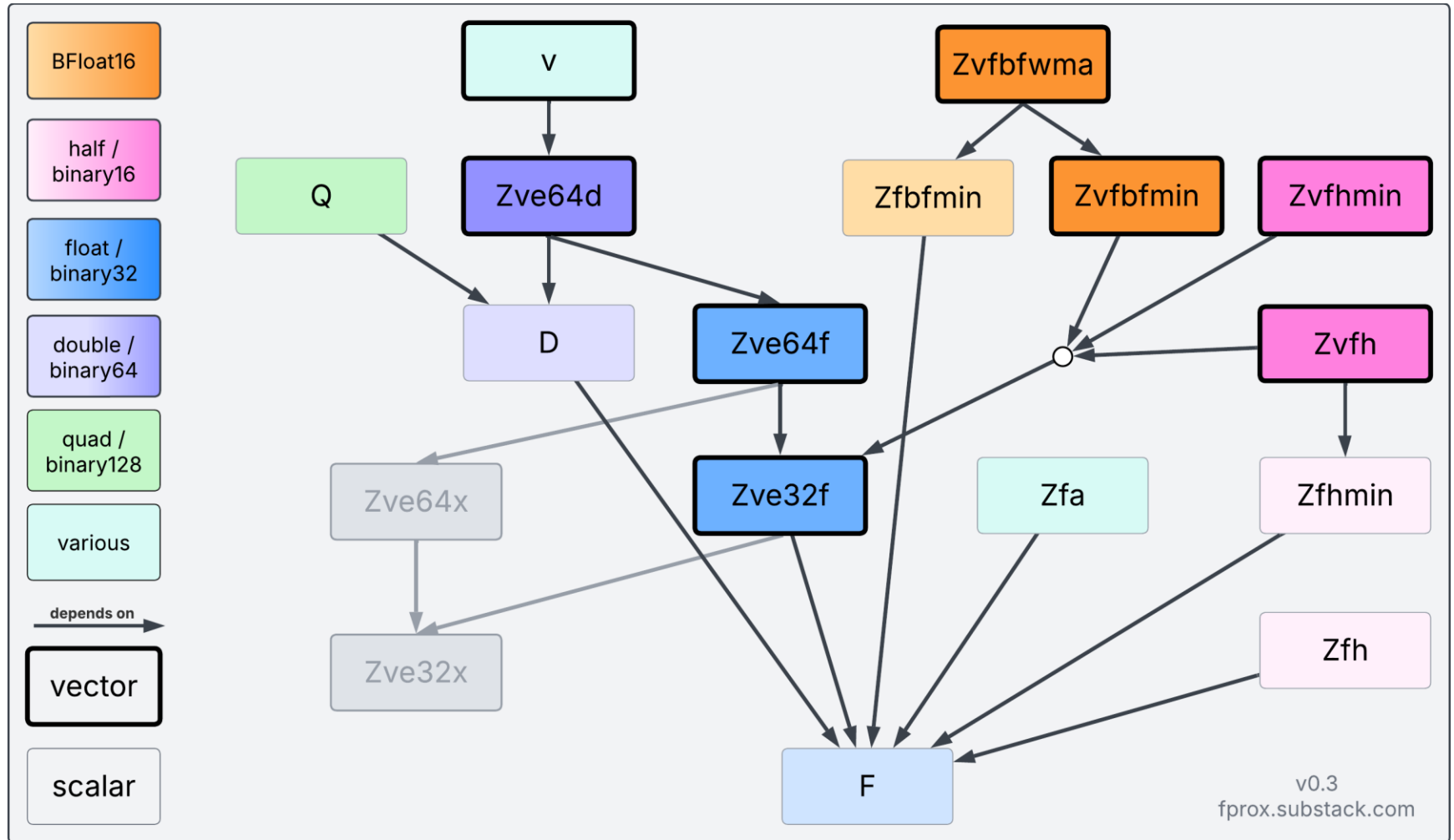
Dual Bachelor in Computer Science and Engineering and Business Administration



Contents

1. Basic concepts on assembly programming
2. RISC-V 32 assembly, memory model and data representation
3. Instruction formats, addressing modes, and instruction sets
4. **Floating point in assembly**
 - ▶ Registers
 - ▶ Data transfer
 - ▶ Arithmetic operations
 - ▶ Detect and compare
 - ▶ Memory
5. Procedure calls and stack convention

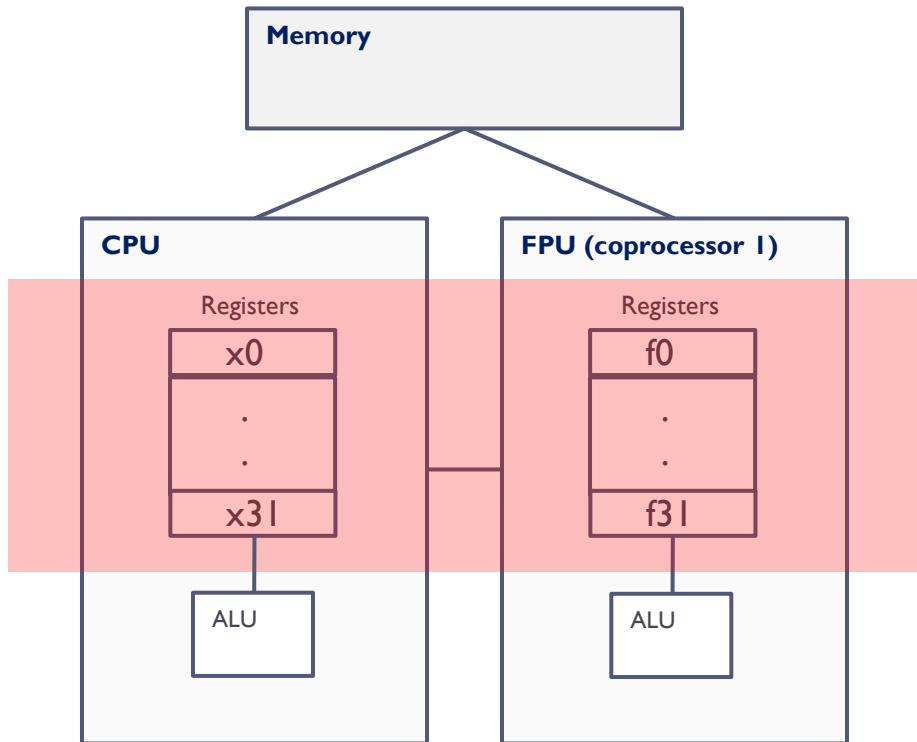
Floating Point in RISC-V



Contents

1. Basic concepts on assembly programming
2. RISC-V 32 assembly, memory model and data representation
3. Instruction formats, addressing modes, and instruction sets
4. Floating point in assembly
 - ▶ Registers
 - ▶ Data transfer
 - ▶ Arithmetic operations
 - ▶ Detect and compare
 - ▶ Memory
5. Procedure calls and stack convention

RISC-V 32



▶ Registers

- ▶ Inside the processor
- ▶ Instructions work with values in registers
- ▶ 32 registers of 32 bits

▶ Memory

- ▶ Outside the processor
- ▶ Data exchange to/from registers
- ▶ More capacity but longer access time than registers
- ▶ 32-bit addresses
 - ▶ 4 GiB addressable

Register file (integer)

summary

Symbolic name	Number	Usage
zero	x0	Constant 0
ra	x1	Return address (used by function calls)
sp	x2	Stack pointer
gp	x3	Pointer to global area
tp	x4	Thread pointer
t0...t2	x5-x7	Temporary (NO preserved across calls)
s0/fp	x8	Temporary (preserved across calls) / Frame pointer
s1	x9	Temporary (preserved across calls)
a0...a1	x10...11	Input function arguments / return values
a2...a7	12...x17	Input function arguments
s2... s11	x18...x27	Temporary (preserved across calls)
t3...t6	x28...x31	Temporary (NO preserved across calls)

- ▶ There are 32 registers
 - ▶ Size: 4 bytes (1 word)
 - ▶ Dual name: logical and numerical (starts with **x** at the beginning)
- ▶ Use convention
 - ▶ Reserved
 - ▶ Arguments
 - ▶ Results
 - ▶ Temporary
 - ▶ Pointers

Register File (floating-point)

ABI Name	Name	Usage
ft0...ft7	f0 ... f7	Temporals (like t...)
fs0...fs1	f8 ... f9	There are saved (like s...)
fa0...fa1	f10 ... f11	Arguments/return (like a...)
fa2...fa7	f12 ... f17	Arguments (like a...)
fs2...fs11	f18 ... f27	There are saved (like s...)
ft8...ft11	f28 ... f31	Temporals (like t...)

- ▶ There are 32 registers
- ▶ The ft0 register does not have its value set to 0.
- ▶ In the single precision extension, the registers are 32 bits (4 bytes).
- ▶ In the double precision extension, the registers are 64 bits (8 bytes) and can store:
 - ▶ Single precision values in the lower 32 bits of the register
 - ▶ Double precision values in the 64 bits of the register

Copy instructions

(floating-point registers $\leftrightarrow$ floating-point registers)

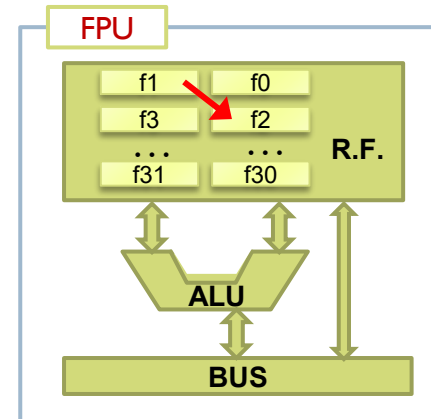
- ▶ Copy data:

- ▶ Between FPU registers
- ▶ Between CPU registers and FPU registers

- ▶ Examples:

- ▶ Copy register to register (.s .d):

`fmv.s f2, f1` *# rd = rs*

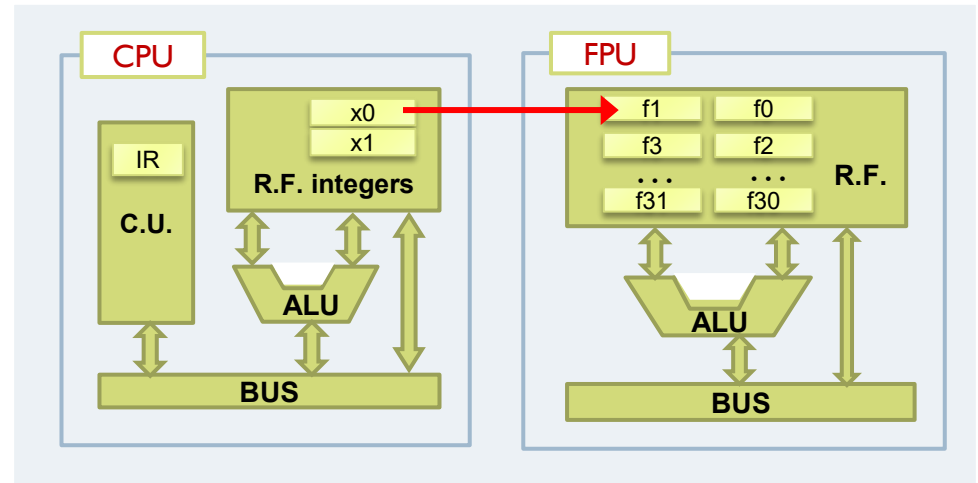


Copy instructions

(integer registers $\leftrightarrow$ floating-point registers)

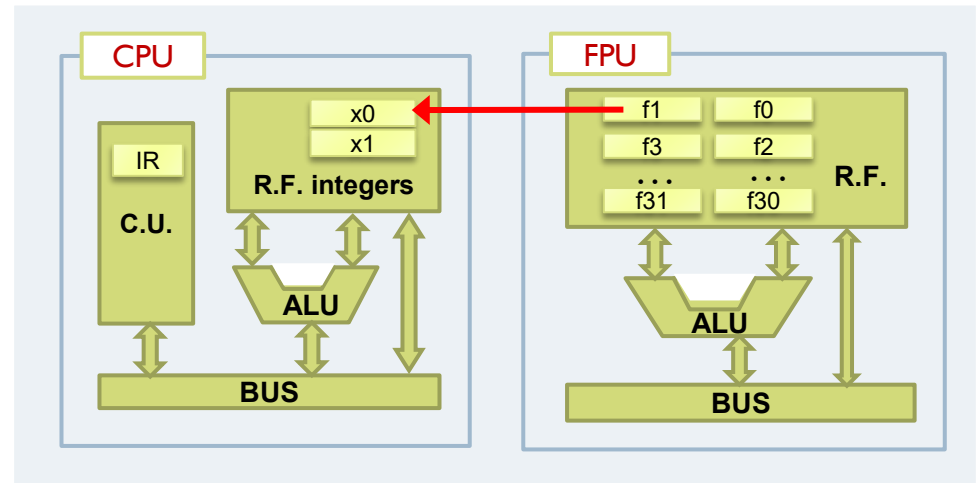
`fmv.w.x rds rsw`

- Copy from integer reg. `rs` to a floating register `rd` (*single precision*)



`fmv.x.w rdw rss`

- Copy from floating register `rs` (*single precision*) to an integer register `rd`



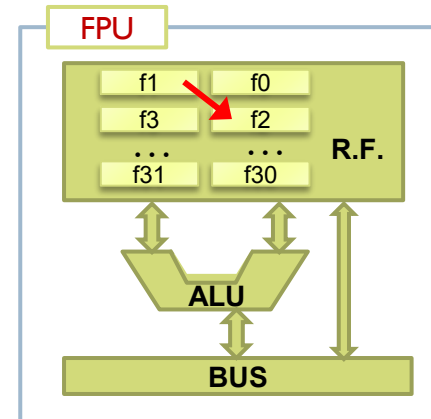
Copy instructions

(floating-point registers $\leftrightarrow$ floating-point registers)

- ▶ It does not exist `fli.s` and `fli.d` (*):
 - ▶ The `Zfa` extension provides `fli.s` and `fli.d` ...
 - ▶ ... But these instructions use a 5-bit index to immediately load one of the 32 most common constant values in scientific and mathematical software (1.0, 0.5, min, inf, etc.)

- ▶ Examples:

- ▶ ~~Immediate load~~
`fli.s` **`f0,`** **`5.0`**



Example

```
float PI = 3.1415;
```

```
.text
```

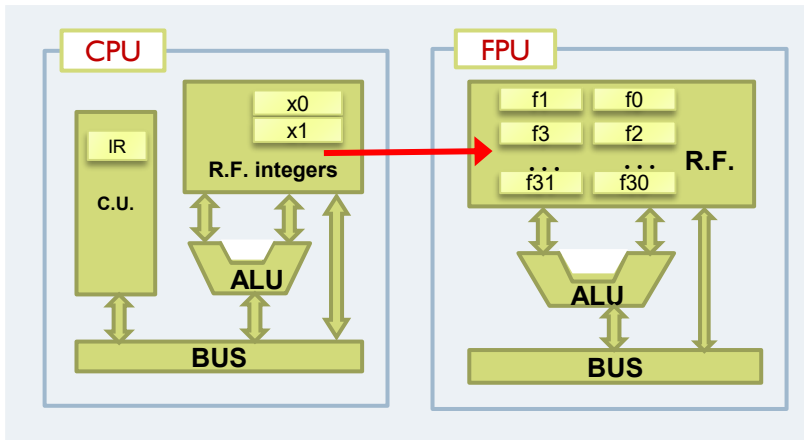
```
main:
```

```
# * It doesn't exist
#   fli.s ft0 3.1415
# * 0x40490E56 is the
#   representation IEEE754
#   in hexadecimal for 3.1415
```

```
li      t0, 0x40490E56
```

```
fmv.w.x ft0, t0
```

```
jr ra
```



Conversion operations (1/3)

double precision $\leftrightarrow$ simple precision

► `fcvt.s.d rd, rs1`

- Convert from double precision (value in floating register `rs`) to simple precision (floating register `rd`).

► `fcvt.d.s rd, rs`

- Convert from simple precision (value in floating register `rs`) to double precision (floating register `rd`).

Conversion operations (2/3)

integer $\leftrightarrow$ simple precision

► `fcvt.w.s rd, rs`

- Convert from simple precision (value in floating register `rs`) to a 32-bit integer 32 **with** sign (integer register `rd`).

► `fcvt.wu.s rd, rs`

- Convert from simple precision (value in floating register `rs`) to a 32-bit integer 32 **without** sign (integer register `rd`).

► `fcvt.s.w rd, rs`

- Convert a 32-bit integer 32 **with** sign (value in integer register `rs`) to a simple precision (floating register `rd`).

► `fcvt.s.wu rd, rs`

- Convert a 32-bit integer 32 **without** sign (value in integer register `rs`) to a simple precision (floating register `rd`).

Conversion operations (3/3)

integer <-> double precision

► `fcvt.w.d rd, rs`

- Convert from double precision (value in floating register `rs`) to a 32-bit integer 32 **with** sign (integer register `rd`).

► `fcvt.wu.d rd, rs`

- Convert from double precision (value in floating register `rs`) to a 32-bit integer 32 **without** sign (integer register `rd`).

► `fcvt.d.w rd, rs`

- Convert a 32-bit integer 32 **with** sign (value in integer register `rs`) to a double precision (floating register `rd`).

► `fcvt.d.wu rd, rs1`

- Convert a 32-bit integer 32 **without** sign (value in integer register `rs`) to a double precision (floating register `rd`).

Example

```
float THREE = 3.0;
```

```
.text
```

```
main:
```

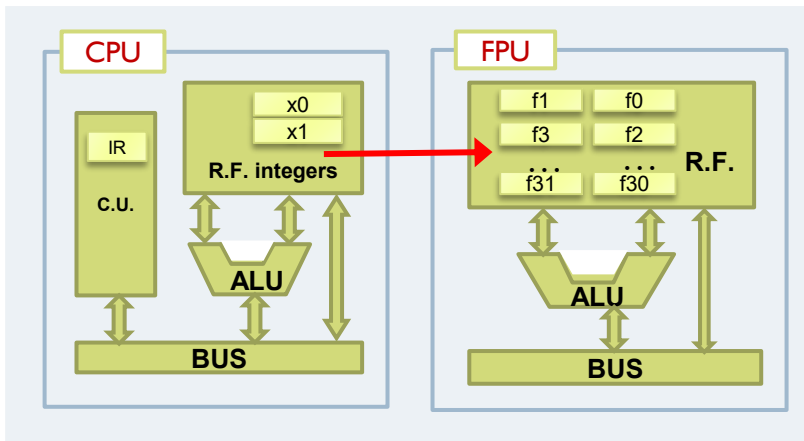
```
# * It doesn't exist
```

```
# fli.s ft0 3.0
```

```
li      t0, 3
```

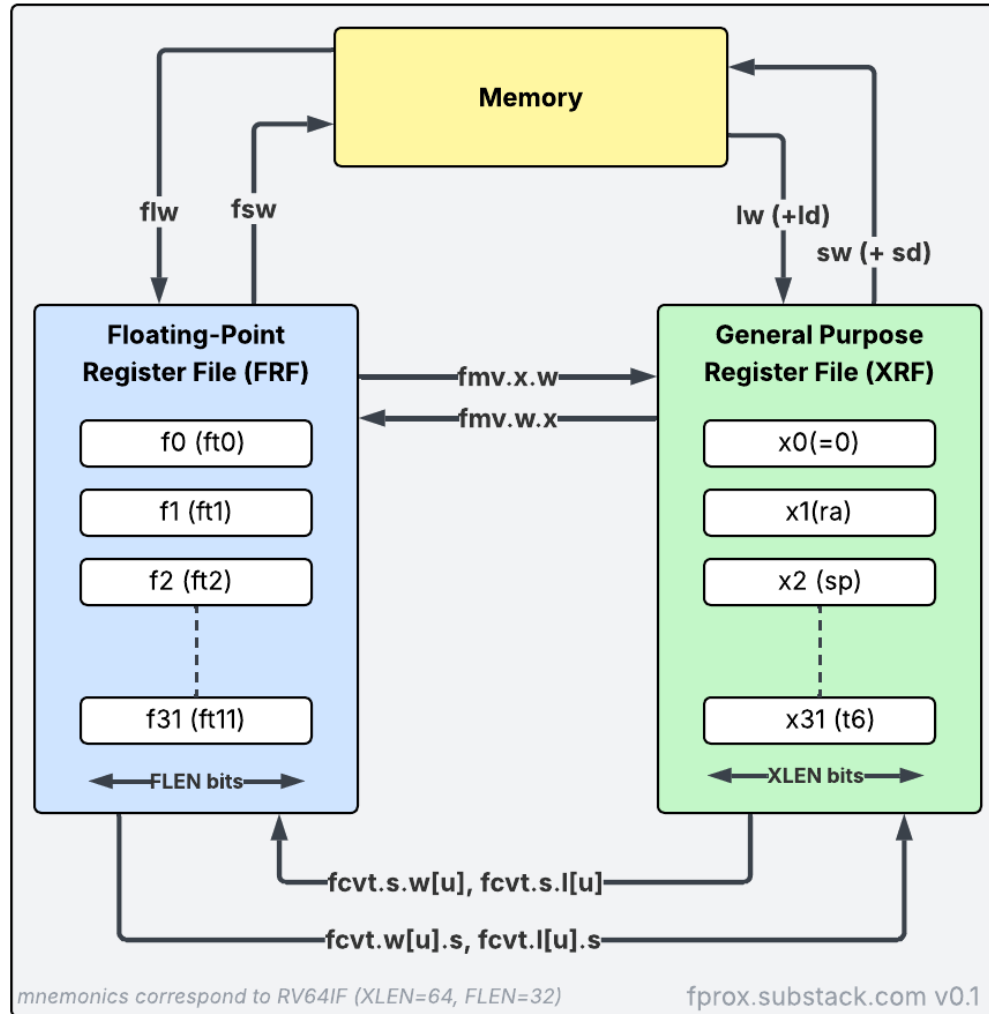
```
fcvt.s.w ft0, t0
```

```
jr ra
```



Move and conversion instructions...

Summary

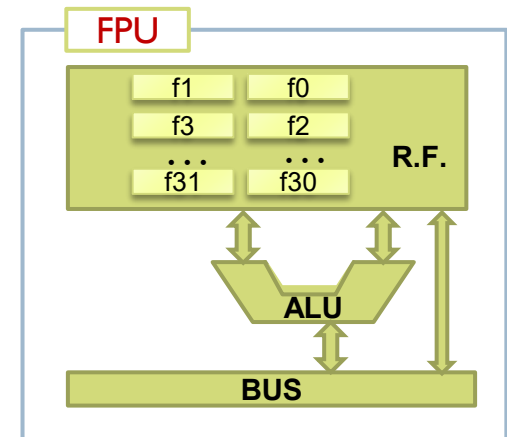


Contents

1. Basic concepts on assembly programming
2. RISC-V 32 assembly, memory model and data representation
3. Instruction formats, addressing modes, and instruction sets
4. **Floating point in assembly**
 - ▶ Registers
 - ▶ Data transfer
 - ▶ **Arithmetic operations**
 - ▶ Detect and compare
 - ▶ Memory
5. Procedure calls and stack convention

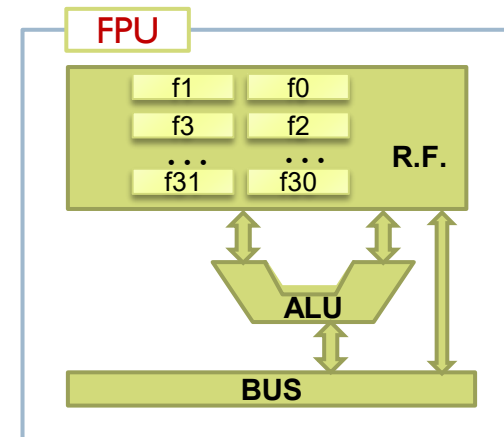
Common arithmetic operations (1/2)

- ▶ Simple (.s) and double (.d) precision:
 - ▶ `fadd.s rd rs1 rs2` $\# \text{rd} = \text{rs1} + \text{rs2}$
 - ▶ `fsub.s rd rs1 rs2` $\# \text{rd} = \text{rs1} - \text{rs2}$
 - ▶ `fmul.s rd rs1 rs2` $\# \text{rd} = \text{rs1} * \text{rs2}$
 - ▶ `fdiv.s rd rs1 rs2` $\# \text{rd} = \text{rs1} / \text{rs2}$
 - ▶ `fabs.s rd rs` $\# \text{rd} = |\text{rs}|$
 - ▶ `fneg.s rd rs` $\# \text{rd} = -\text{rs}$



Common arithmetic operations (2/2)

- ▶ Simple (.s) and double (.d) precision:
 - ▶ `fmin.s rd rs1 rs2` # $rd = \min(rs1, rs2)$
 - ▶ `fmax.s rd rs1 rs2` # $rd = \max(rs1, rs2)$
 - ▶ `fsqrt.s rd rs1` # $rd = \sqrt{rs1}$
 - ▶ `fmadd.s rd rs1 rs2 rs3` # $rd = rs1 \times rs2 + rs3$
 - ▶ `fmsub.s rd rs1 rs2 rs3` # $rd = rs1 \times rs2 - rs3$



Example

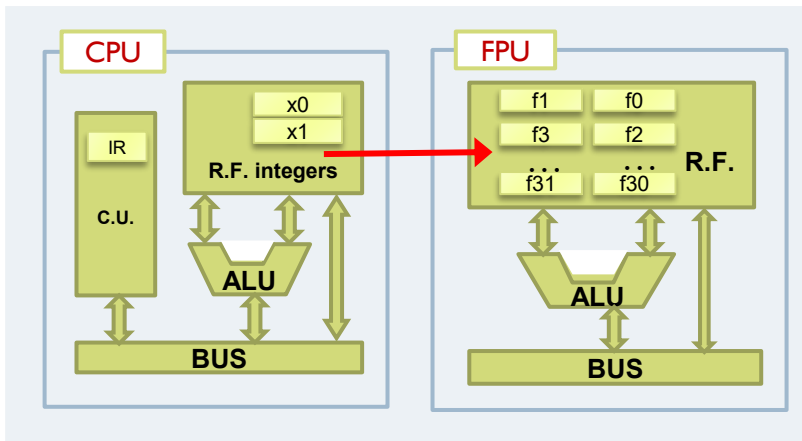
```
float PI    = 3,1415;
int  radio = 4;
float length;

length = PI * radio;
```

```
.text
main:
```

```
li      t0, 0x40490E56
fmv.w.x ft0, t0    # ft0 ← t0
li      t1,  4     # 4 in Ca2
fcvt.s.w ft1, t1    # 4 in ieee754
fmul.s  ft0, ft0, ft1
```

```
jr ra
```



Contents

1. Basic concepts on assembly programming
2. RISC-V 32 assembly, memory model and data representation
3. Instruction formats, addressing modes, and instruction sets
4. **Floating point in assembly**
 - ▶ Registers
 - ▶ Data transfer
 - ▶ Arithmetic operations
 - ▶ **Detect and compare**
 - ▶ Memory
5. Procedure calls and stack convention

Classification of floating-point numbers

- ▶ `fclass.s rd, rs1` (simple precision)
- ▶ `fclass.d rd, rs1` (double precision)
- ▶ Save in `rd` the type of the floating-point number of the register `rs1`:

Bit in rd	Meaning
0	-Inf
1	Negative normalized number
2	Negative non-normalized number
3	-0
4	+0
5	Positive non-normalized number
6	Positive normalized number
7	+Inf
8	NaN (non-quiet)
9	NaN (quiet)

IEEE Standard 754 (single precision)

summary

$$(-1)^s \times 0.\text{mantissa} \times 2^{-126}$$

Exponent	Mantissa	Special value
0 (0000 0000)	0	+/- 0 (depends on sign)
0 (0000 0000)	$\neq 0$	Number NOT normalized
255 (1111 1111)	$\neq 0$	NaN (0/0, sqrt(-4),)
255 (1111 1111)	0	+/- infinite (depends on sign)
1-254	Any	Normalized number (no special)

$$(-1)^s \times 1.\text{mantissa} \times 2^{\text{exponent}-127}$$

Example

```
float PI = 3.1415;
```

```
print(typeof(PI));
```

```
.text
```

```
main:
```

```
# ft0 ← 3.0
```

```
li      t0, 3
```

```
fcvt.s.w ft0, t0
```

```
fclass.s t1, ft0
```

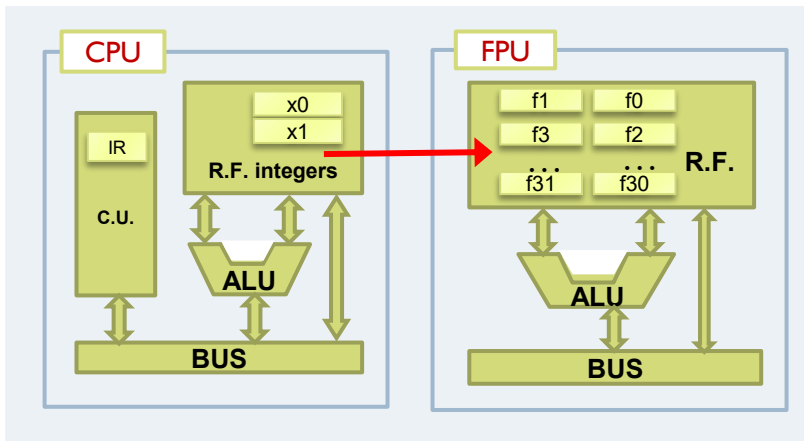
```
# print(typeof(ft0))
```

```
li a7, 1
```

```
mv a0, t1
```

```
ecall
```

```
jr ra
```



Comparison instructions (floating-point registers)

► Simple precision:

- `feq.s rd, rs1, rs2` if ($rs1 == rs2$) $rd = 1$; else $rd = 0$
- `fle.s rd, rs1, rs2` if ($rs1 \leq rs2$) $rd = 1$; else $rd = 0$
- `flt.s rd, rs1, rs2` if ($rs1 < rs2$) $rd = 1$; else $rd = 0$

► Doble precision:

- `feq.d rd, rs1, rs2` if ($rs1 == rs2$) $rd = 1$; else $rd = 0$
- `fle.d rd, rs1, rs2` if ($rs1 \leq rs2$) $rd = 1$; else $rd = 0$
- `flt.d rd, rs1, rs2` if ($rs1 < rs2$) $rd = 1$; else $rd = 0$

Branches with floating-point numbers

```
x = 3.0;
```

```
y = 2.0;
```

```
if (x < y)
```

```
{
```

```
    . . .
```

```
}
```

```
.text
```

```
main:
```

```
li      t0, 3
```

```
fcvt.s.w ft0, t0
```

```
li      t1, 2
```

```
fcvt.s.w ft1, t1
```

```
flt.s t0, ft1, ft2
```

```
bne t0, x0, end_if1
```

```
# . . .
```

```
end_if1:
```

```
jr ra
```

Contents

1. Basic concepts on assembly programming
2. RISC-V 32 assembly, memory model and data representation
3. Instruction formats, addressing modes, and instruction sets
4. Floating point in assembly
 - ▶ Registers
 - ▶ Data transfer
 - ▶ Arithmetic operations
 - ▶ Detect and compare
 - ▶ Memory
5. Procedure calls and stack convention

Representation of basic data types (1/3)

```
// boolean
bool_t b1 ;
bool_t b2 = false ;
```

```
// character
char c1 ;
char c2 = 'x' ;
```

```
// integers
int res1 ;
int op1 = -10 ;
```

```
// floating point
float f0 ;
float f1 = 1.2 ;
double d2 = 3.0e10 ;
```

```
.data

# boolean
b1:    .zero    1           # 1 bytes
b2:    .byte    0           # 1 byte

# character
c1:    .byte           # 1 bytes
c2:    .byte 'x'       # 1 bytes

# integers
res1:  .zero    4           # 4 bytes
op1:   .word    -10         # 4 bytes

# floating point
f0:    .float           # 4 bytes
f1:    .float    1.2       # 4 bytes
d2:    .double   3.0e10    # 8 bytes
```

Definition of static data (at compile time)

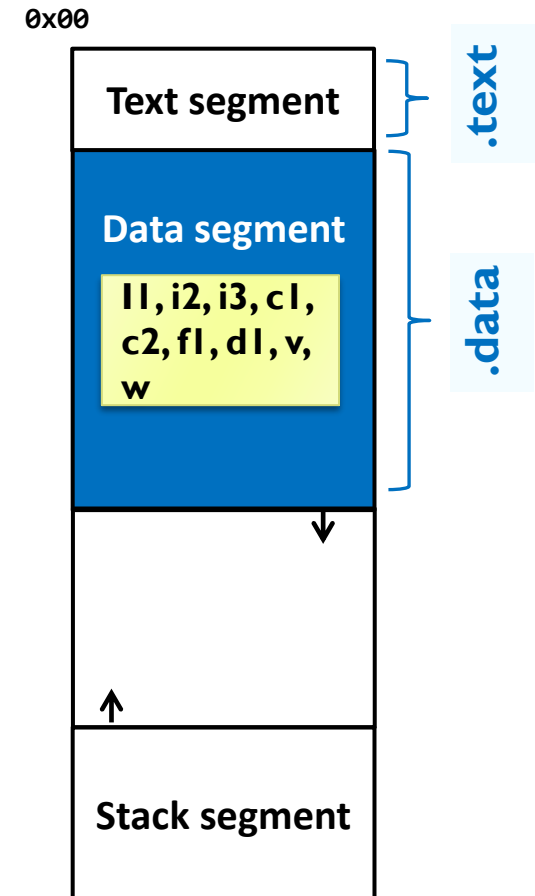
label (address)

Data type (directive)

value

```
.data
cadena: .string "Hello world\n"
i1: .word 10      # int    i1=10
i2: .word -5      # int    i2=-5
i3: .half 300     # short  i3=300
c1: .byte 100     # char   c1=100
c2: .byte 'a'     # char   c2='a'
f1: .float 1.3e-4 # float  f1=1.3e-4
d1: .double .001  # double d1=0.001

# int v[3]={0,-1,0xffffffff}; int w[100];
v: .word 0, -1, 0xffffffff
w: .zero 400
```



Memory operations: floating point

Memory access (floating point)		
<code>flw rs, 10(rs1)</code>	<code>rs = Memory[rs1+10]</code>	Loads the single precision value stored at address (rs1+10) into the floating-point register rd.
<code>fsw rs, 10(rs1)</code>	<code>Memory[rs1+10] = rs</code>	Stores the single value precision of the rs register at address (rd1+10).
<code>fld rd, 10(rs1)</code>	<code>rd = Memory[rs1+10]</code>	Loads the double precision value stored at address (rs1+10) into the rd register.
<code>fsd rd, 10(rs1)</code>	<code>Memory[rs1+10] = rd</code>	Stores the double precision value of the rs register at address (rd1+10).

Elemental data types

Float

```
float  result ;
float  op1 = 100 ;
float  op2 = 2.5
```

...

```
main ()
```

```
{
    result = op1 + op2 ;
    ...
}
```

.data

```
.align 2
result: .zero 4 # 4 bytes
op1:    .float 100
op2:    .float 2.5
# ...
```

.text

```
main:  la      t0,  op1
        flw     ft0, 0(t0)
        la      t0,  op2
        flw     ft1, 0(t0)
        fadd.s  ft3, ft1, ft2
        la      t0,  result
        fsw     ft3, 0(t0)
# ...
```

```
jr ra
```

Elemental data types

Double

```
double result ;
double op1 = 100 ;
double op2 = -10.27 ;
...
```

```
main ()
{
    result = op1 + op2 ;
    ...
}
```

.data

```
.align 2
result: .zero 8 # 8 bytes
op1:    .double 100
op2:    .double 2.5
# ...
```

.text

```
main:  la      t0,  op1
        fld     ft0, 0(t0)
        la      t0,  op2
        fld     ft1, 0(t0)
        fadd.s  ft3, ft1, ft2
        la      t0,  result
        fsd     ft3, 0(t0)
# ...
```

```
jr ra
```

Example. Calculate

$$\sum_{i=1}^N \frac{1}{2^i}$$

```
.data

N:          .word  10           # Upper limit
one_f:      .float 1.0         # 1.0 in floating-point format
two_f:      .float 2.0         # 2.0 in floating-point format

.text

main:
    la      t0, N
    lw      t0, 0(t0)          # t0 = N = 10

    la      t1, one_f
    flw     ft0, 0(t1)         # ft0 = 1.0 (numerator)

    la      t1, two_f
    flw     ft1, 0(t1)         # ft1 = 2.0 (base of the power)

    fmv.w.x fa0, zero          # accumulates the result

    # Initialize the variables for the loop
    li      t1, 1              # t1 = i = 1
    fmv.s   ft2, ft1           # ft2 = 2.0 (will represent 2^i, starting at 2^1)

    # to be continued...
```

Example. Calculate

$$\sum_{i=1}^N \frac{1}{2^i}$$

```
loop:

    bgt t1, t0, end_loop

    # Calculate current term: ft3 = 1.0 / (2^i)
    fdiv.s ft3, ft0, ft2 # ft3 = ft0 / ft2

    # Add to the total accumulator: fa0 = fa0 + ft3
    fadd.s fa0, fa0, ft3

    # Update for the next iteration
    fmul.s ft2, ft2, ft1 # Multiplies the power by 2 for the next power (2^(i+1))
    addi t1, t1, 1      # i = i + 1

j loop

end_loop:

# Print the result
li a7, 2
ecall

li a7, 10    # syscall 10 = exit
ecall
```

Exercise

- ▶ Write a program that:
 - ▶ Loads the value -3.141516 into register f0
 - ▶ Allows you to retrieve the value of the exponent and the mantissa stored in register f0 (in IEEE 754 format)
 - ▶ Prints the sign
 - ▶ Prints the exponent
 - ▶ Prints the mantissa

Exercise (solution)

```
.data
    valor:    .float -3.141516
```

```
.text
```

```
main:
```

```
    la    t0, valor
    flw   ft0, 0(t0)
```

```
    #se imprime
```

```
    fmv.s fa0, ft0
```

```
    li    a7, 2
```

```
    ecall
```

```
    li    a0, '\n'    # enter
```

```
    li    a7, 11
```

```
    ecall
```

```
    # copy to CPU register
```

```
    fmv.x.w    t0,    ft0
```

```
    li    t1, 0x80000000    # sign
    and    a0, t0, t1
    srli    a0, a0, 31
    li    a7, 1
    ecall
```

```
    li    a0, '\n'          # enter
    li    a7, 11
    ecall
```

```
    li    t1, 0x7F800000    # exponent
    and    a0, t0, t1
    srli    a0, a0, 23
    li    a7, 1
    ecall
```

```
    li    a0, '\n'          # enter
    li    a7, 11
    ecall
```

```
    li    t1, 0x007FFFFF    # mantissa
    and    a0, t0, t1
    li    a7, 1
    ecall
```

```
    jr    ra
```

ARCOS Group

uc3m | Universidad **Carlos III** de Madrid

L3: Fundamentals of assembler programming (4) **Computer Structure**

Bachelor in Computer Science and Engineering

Bachelor in Applied Mathematics and Computing

Dual Bachelor in Computer Science and Engineering and Business Administration

